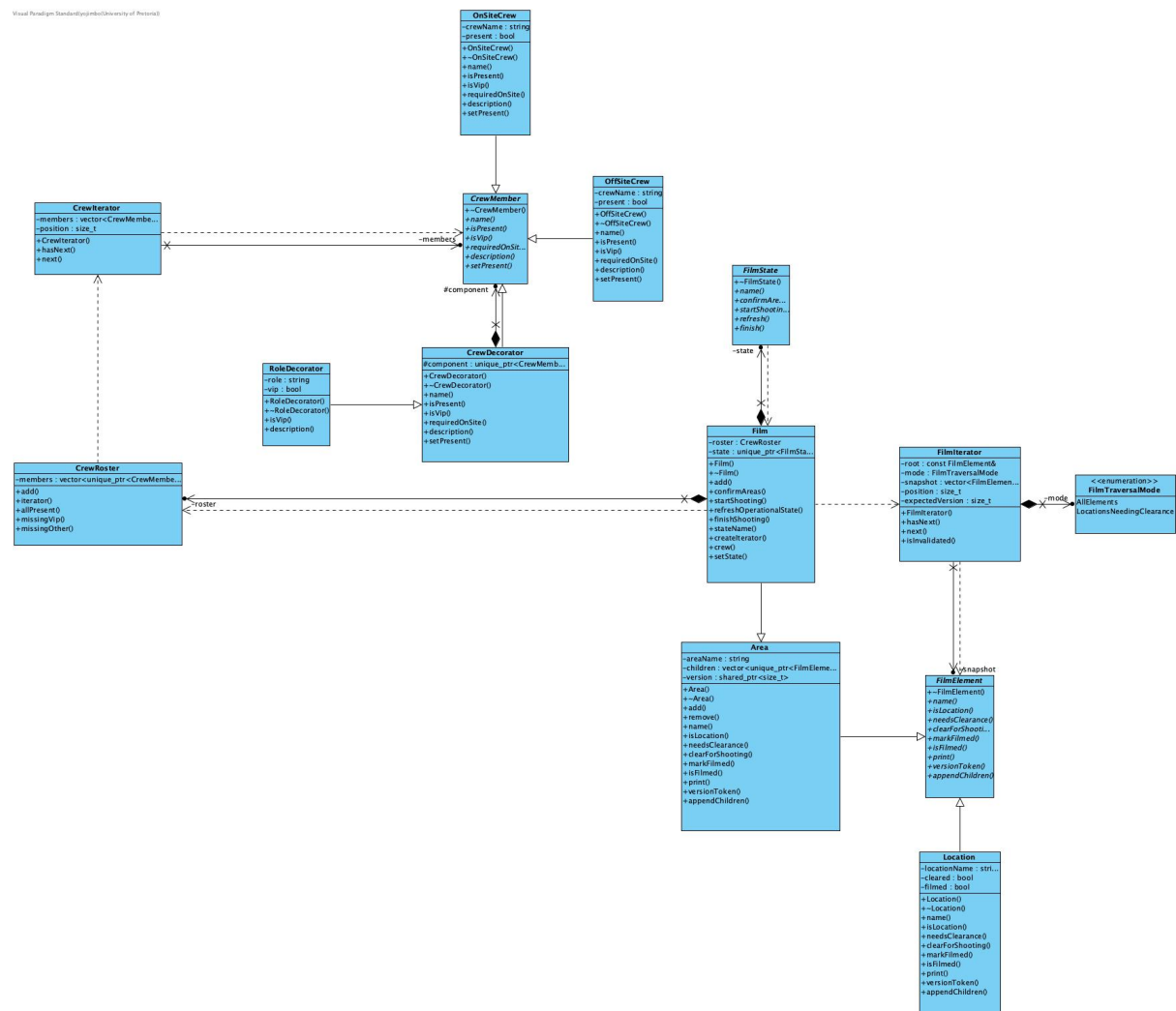


JA Rafudeen – u25011503
AH Ramaboea - u25445392
YH Smit – u25433726

Task 1

We have chosen Film Production as our domain. When directing a large-scale film, there are many moving parts, clearance for public areas need to be made, every crew member needs to be on time, all necessary equipment needs to be on set and ready, etc. Our program aims to assist with this by helping the Assistant Director in organizing everything, finding replacements, and other such actions which will increase the efficiency of filming and minimize downtime.



Composite participants:
 Component – FilmElement
 Composite – Area, Film
 Leaf – Location

Iterator participants:

ConcreteIterator – FilmIterator, CrewIterator

ConcreteAggregate – Film, CrewRoster

Iterator and Aggregate – not implemented, as we do not have multiple concrete classes so there is no need for an abstract class.

Decorator participants:

Component – CrewMember

ConcreteComponent – OnSiteCrew, OffSiteCrew

Decorator – CrewDecorator

ConcreteDecorator – RoleDecorator

State participants:

State – FilmState

ConcreteState – NotShootingState, ReadyForShootState, ShootingState,

OtherMissingState, CancelledState, FinishedState

Context - Film

Design Rationale

An area may contain another area, such as a street being inside of a city, and this is especially relevant when acquiring clearance. For example, if we want to block off a road to film a racing scene, we need to know which city/district the road falls under such that we can contact their Metro Police Department to make clearance. This is why we chose area for our **Composite** pattern.

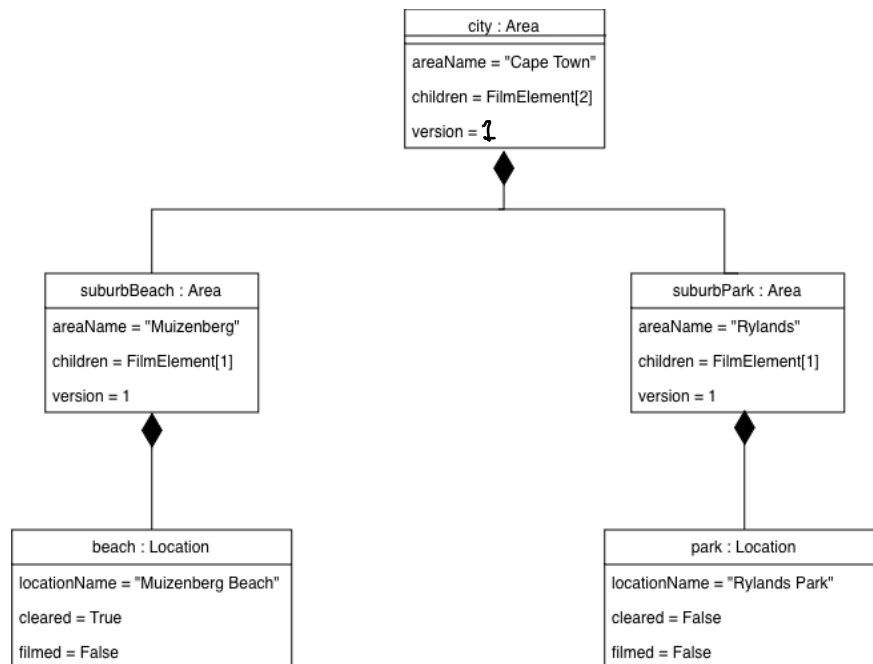
The role of each crew member is vital information to us, as if a crew member is unable to make it, we need to assess how valuable they are and if they can be replaced in order to proceed with shooting. For example, the Director is a VIP and cannot be replaced, whereas a background extra can easily be replaced. Since one crew member can have a mixture of roles, we chose this to be our **Decorator** pattern.

To check for the aforementioned areas and crew members, we use the **Iterator** pattern, allowing us to easily move through each area/member to check for attributes that dictate whether we can begin filming or not.

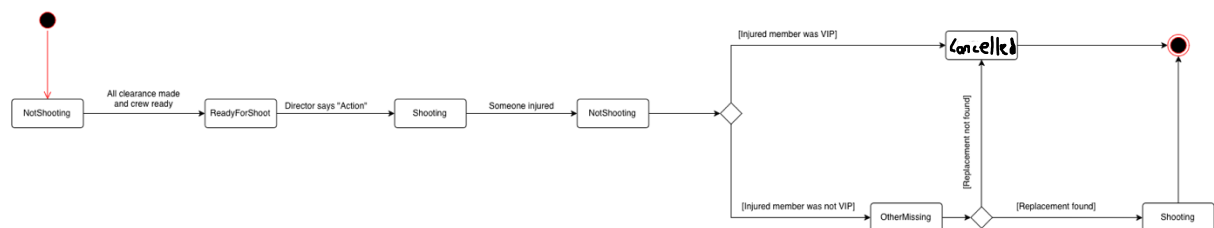
Attempting to film a scene will have different results based on if we're ready to shoot, if we're waiting for clearance, if we need a replacement, etc. To account for different behaviour depending on our situation, we use the **State** pattern.

Task 4

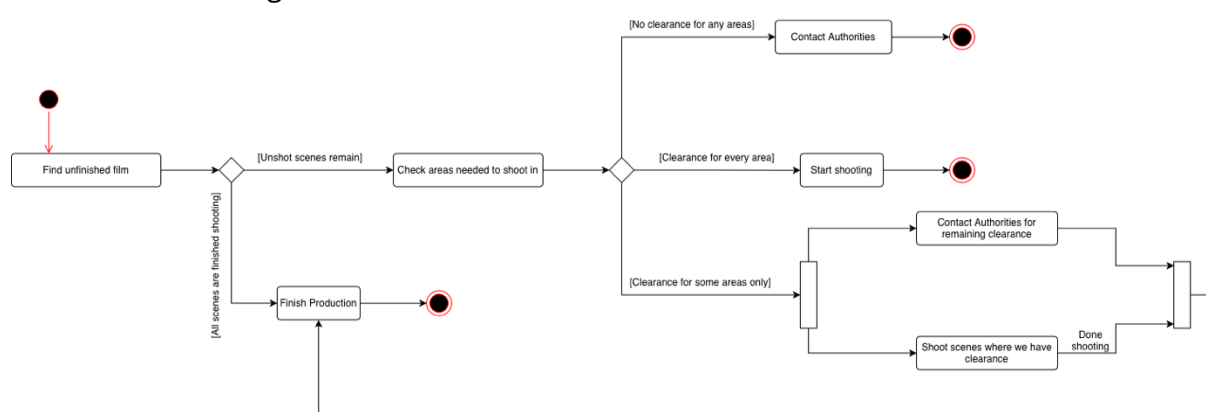
Object diagram:



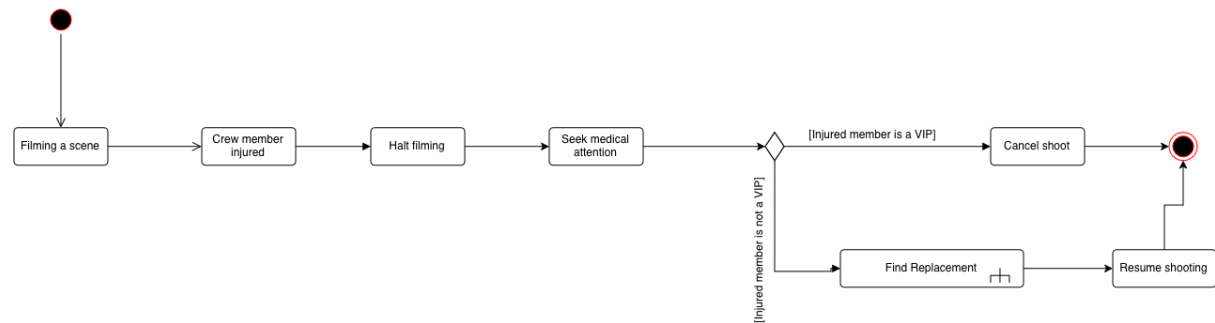
State diagram – models the state changes that occur when a crew member is injured:



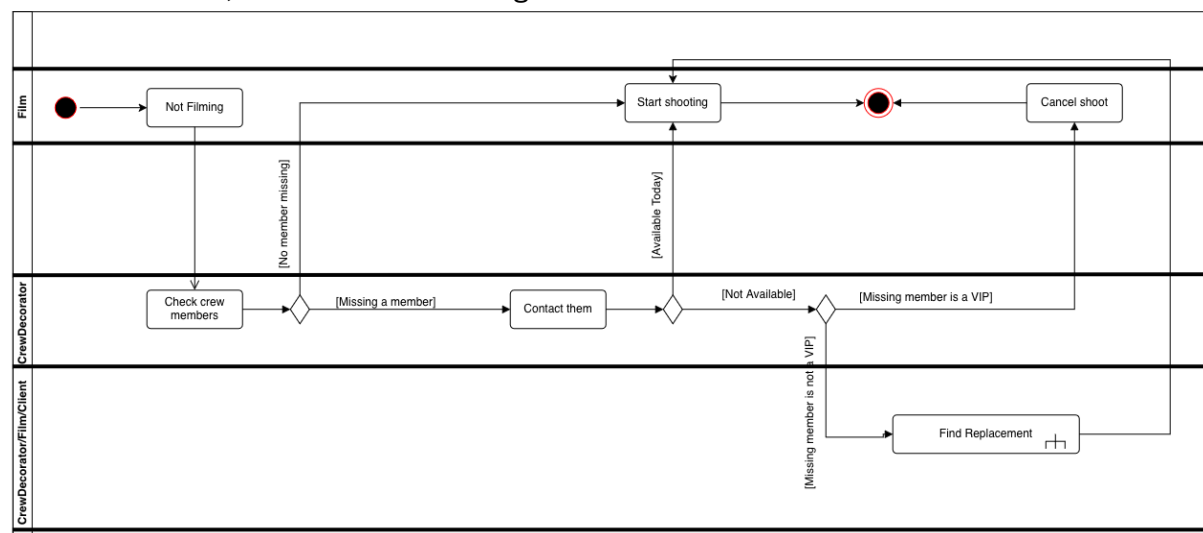
Activity Diagram 1 – traversal visible as part of a larger workflow – finding a film which is unfinished, and if any scenes remain, obtain clearance for them if not obtained already, then finish shooting:



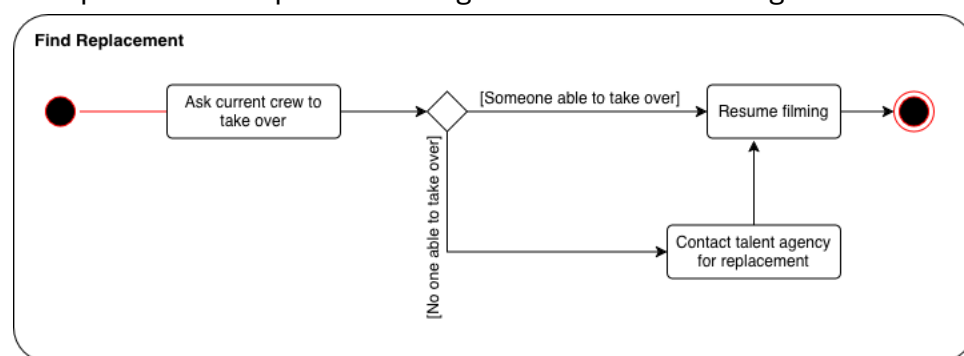
Activity Diagram 2 – meaning conditional behaviour related to runtime configuration of object – models what happens during filming when someone is unexpectedly injured:



Activity Diagram 3 – before filming begins, first checks if all necessary crew members are present, and starts shooting if so. If not, then it finds a replacement, unless the member is a VIP, in which case filming is cancelled:

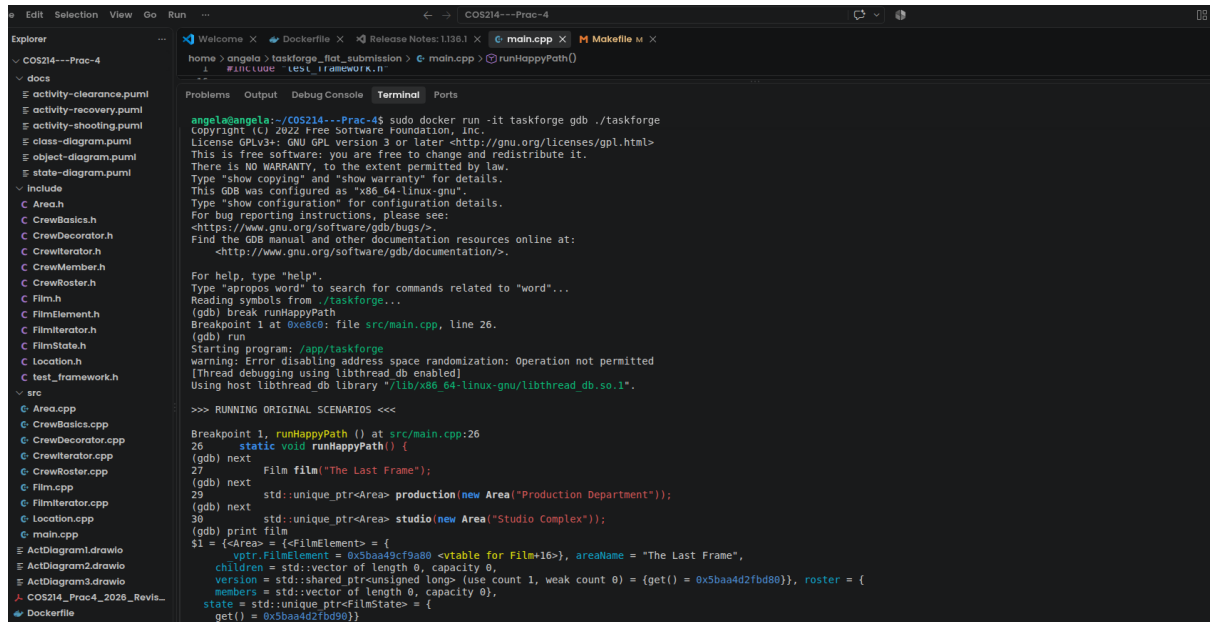


Composite Find Replacement algorithm used in the diagrams:



Task 5

GDB evidence:



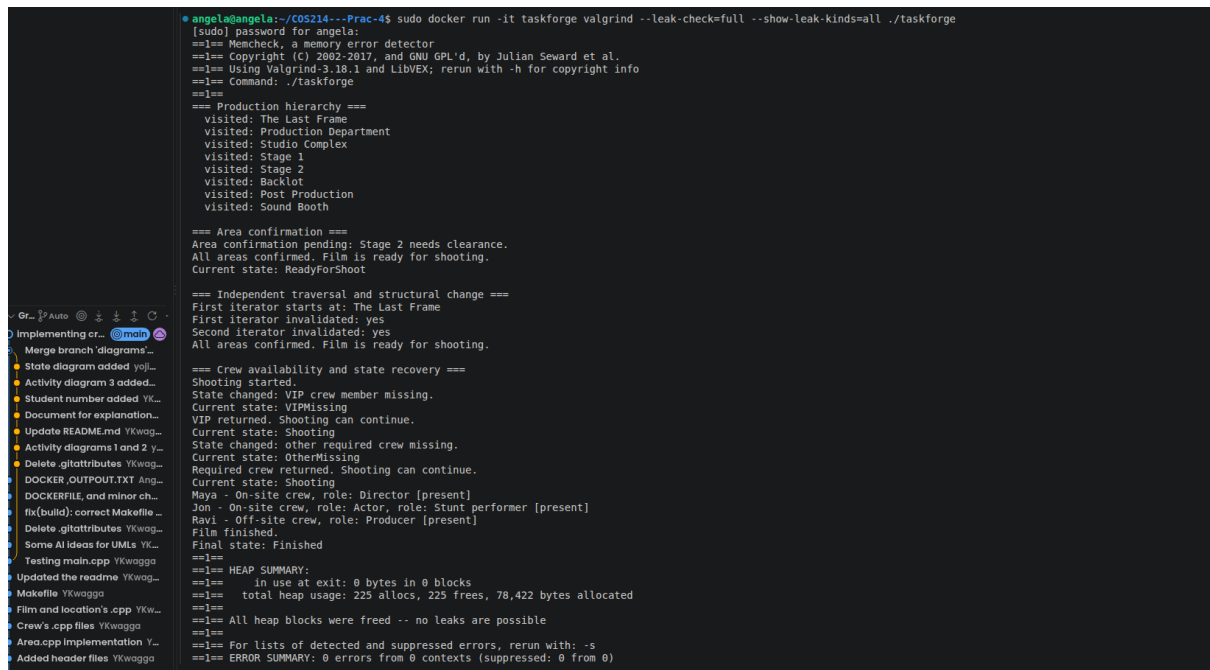
```
angela@angela:~/COS214---Prac-4$ sudo docker run -it taskforge gdb ./taskforge
Copyright (C) 2022 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./taskforge...
(gdb) break runHappyPath
Breakpoint 1 at 0xe8c0: file src/main.cpp, line 26.
(gdb) run
Starting program: /app/taskforge
warning: Error disabling address space randomization: Operation not permitted
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

>>> RUNNING ORIGINAL SCENARIOS <<<

Breakpoint 1, runHappyPath () at src/main.cpp:26
26   static void runHappyPath() {
(gdb) next
27       Film film("The Last Frame");
(gdb) next
29       std::unique_ptr<Area> production(new Area("Production Department"));
(gdb) next
30       std::unique_ptr<Area> studio(new Area("Studio Complex"));
(gdb) print film
$1 = {<Area> = {<FilmElement> = {
  vptr.FilmElement = 0x5baa49cf9a80 <vtable for Film+16>,
  children = std::vector of length 0, capacity 0,
  version = std::shared_ptr<unsigned long> (use count 1, weak count 0) = (get()) = 0x5baa4d2fbd80},
  roster = {
    members = std::vector of length 0, capacity 0,
    state = std::unique_ptr<FileState> = {
      get() = 0x5baa4d2fbd90}}}}
```

Valgrind evidence:



```
angela@angela:~/COS214---Prac-4$ sudo docker run -it taskforge valgrind --leak-check=full --show-leak-kinds=all ./taskforge
[sudo] password for angela:
==1== Memcheck, a memory error detector
==1== Copyright (C) 2002-2017, and GNU GPL'd, by Julian Seward et al.
==1== Using Valgrind-3.18.1 and LibVEX; rerun with -h for copyright info
==1== Command: ./taskforge
==1==

=== Production hierarchy ===
visited: The Last Frame
visited: Production Department
visited: Studio Complex
visited: Stage 1
visited: Stage 2
visited: Backlot
visited: Post Production
visited: Sound Booth

=== Area confirmation ===
Area confirmation pending: Stage 2 needs clearance.
All areas confirmed. Film is ready for shooting.
Current state: ReadyForShoot

=== Independent traversal and structural change ===
First iterator starts at: The Last Frame
First iterator invalidated: yes
Second iterator invalidated: yes
All areas confirmed. Film is ready for shooting.

=== Crew availability and state recovery ===
Shooting started.
State changed: VIP crew member missing.
Current state: VIPMissing
VIP returned. Shooting can continue.
Current state: Shooting
State changed: other required crew missing.
Current state: OtherMissing
Required crew returned. Shooting can continue.
Current state: Shooting
Maya - On-site crew, role: Director [present]
Jon - On-site crew, role: Actor, role: Stunt performer [present]
Ravi - Off-site crew, role: Producer [present]
Film finished.
Final state: Finished
==1==
==1== HEAP SUMMARY:
==1==    in use at exit: 0 bytes in 0 blocks
==1==    total heap usage: 225 allocs, 225 frees, 78,422 bytes allocated
==1==
==1== All heap blocks were freed -- no leaks are possible
==1==
==1== For lists of detected and suppressed errors, rerun with: -s
==1== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Genuine bug:



```
static void printTraversal(FilmIterator iterator) {
    std::cout << " visited: " << iterator.next()->name() << "\n";
}

static void runHappyPath() {
    //Film emptyFilm("Empty");
    //CrewIterator emptyCrew = emptyFilm.crew().iterator();
    //emptyCrew.next(); error encountered here

    Film film("The Last Frame");
```

Task 6

<https://github.com/YKwagga/COS214---Prac-4>

Work was divided based on our strengths and weaknesses, although every member discussed and worked on every aspect of the practical, our main designation was as follows:

One member was in charge of the designing

One member was in charge of the diagrams

One member was in charge of the coding

Github helped us to push the most recent version of our design/diagrams, such that the other team members could remain updated and create the code accordingly. It also helped us in the form of Pull Requests, as different team members could work on the code concurrently and then use a PR to merge them together, allowing us to work more efficiently without sacrificing the integrity of our design.